

Construção ao vivo do vetor de contexto no preditor de intenção: comportamento em malha fechada nas sessões gravadas e escalabilidade (documentação interna)*

Marcos Kalile Meira de Vasconcellos
Orientador: Prof. Hebert Azevedo-Sá, Ph.D.

16 de julho de 2026

Resumo

Os relatórios anteriores mediram o preditor de intenção com o vetor de contexto *ideal* – reconstruído a partir das anotações (*teacher-forced*). Este documento fecha a malha: roda a lógica de `run_webcam_context.py` sobre as 8 sessões gravadas, com o vetor de contexto construído **ao vivo pelo próprio preditor** (intenções confirmadas realimentam o `PlanGraph`), e mede se o modelo treinado *offline* se sustenta nesse regime – e se é escalável. O achado principal: com dois ajustes mínimos (o evento `begin_four_tubes` vindo da anotação gravada, como substituto do comando de voz; e o aumento da janela de confirmação de 3 para 8), o custo de construir o contexto ao vivo é pequeno – **top-1 de teste 68,5% ao vivo contra 70,9% com contexto ideal** (−2,4 p.p.), com erros de transição do `PlanGraph` zerados no teste. Restam duas fragilidades concretas, medidas e reportadas: uma *deriva temporal* (a acurácia cai ao longo da sessão) e a classe `get_wheels`, cujo *recall* despenca de 97% (ideal) para 62% (ao vivo) por confusão com `get_screws`. Todas as métricas excluem os trechos rotulados `ignore`. As matrizes de confusão seguem o padrão do repositório `hrc-finetune`.

Conteúdo

1	Introdução	2
1.1	Contexto e o que muda neste relatório	2
1.2	Como a malha fechada é reproduzida	2
2	Métricas usadas	2
3	Protocolo	3
3.1	Dados e variante	3
3.2	Dois ajustes mínimos, baseados no repositório original	3
3.3	Exclusão dos trechos <code>ignore</code>	4

*Nota de proveniência: este relatório e o *pipeline* de análise que o sustenta (repositório `hrc-live-context-eval`) foram produzidos com assistência de IA (Claude, Anthropic), a pedido do autor. Diferente dos relatórios anteriores, os números aqui *não* vêm de uma experimentação manual prévia: eles são a saída da execução desse *pipeline* de análise sobre as sessões gravadas e os *checkpoints* já treinados pelo autor (`hrc-finetune`, variante V2 com contexto 10D). Nenhum arquivo dos repositórios originais foi modificado. O código, as tabelas e as figuras são reproduzíveis via `scripts/analyze_live.py`.

4	Resultados	4
4.1	Ajuste da janela de confirmação	4
4.2	Desempenho ao vivo vs. contexto ideal	4
4.3	Escalabilidade	5
4.4	Onde o contexto erra, e o papel do comando de voz	6
4.5	Integridade das confirmações por sessão	7
5	Discussão e recomendações	8
6	Conclusão	8

1 Introdução

1.1 Contexto e o que muda neste relatório

O projeto adapta o preditor de intenção humano-robô de Yu et al. [1] para captura por webcam (pose via MediaPipe) e injeta um *vetor de contexto* – um resumo numérico do progresso da montagem (peças de cada tipo já consumidas, etapa atual do grafo de tarefa) – para melhorar a predição da próxima peça a ser pega (classes `no_action`, `get_connectors`, `get_screws`, `get_wheels`). Relatórios anteriores mostraram que esse contexto melhora muito a acurácia (ganho de até +30,9 p.p. de top-1) – *mas sempre avaliando com o vetor de contexto correto*, calculado a partir das anotações manuais de cada sessão.

Numa aplicação real não há anotação: o vetor de contexto tem de ser **construído ao vivo**, e a única fonte de “o que já foi feito” são as próprias predições do modelo. É essa a questão deste relatório: *quando o modelo dirige a construção do seu próprio contexto (malha fechada), ele se sustenta? É escalável? Se não, o que fazer?* A resposta importa porque o vetor de contexto é cumulativo: uma predição errada, se confirmada, corrompe o estado e todas as predições seguintes recebem um contexto errado.

1.2 Como a malha fechada é reproduzida

Reproduzimos a lógica do controlador ao vivo (`run_webcam_context.py` do `hrc-finetune`) sobre as sessões gravadas, lendo o `skeleton.pkl` quadro a quadro (sem câmera). O contexto começa a frio (`PlanGraph` recém-instanciado) e é mutado **apenas quando uma intenção é confirmada**: uma predição de ação só vira uma confirmação depois de se repetir em `send_window` quadros consecutivos – uma janela de confirmação que protege o estado cumulativo de picos isolados de ruído. Cada confirmação chama `plan.step(intenção)`, que atualiza os contadores e o vetor de contexto usado nas predições seguintes. Nenhuma anotação de intenção é usada na malha; as anotações entram só como *gabarito* para medir o resultado.

2 Métricas usadas

Três medidas guiam a análise; todas excluem os quadros `ignore` (Seção 3.3).

- **Acurácia *streaming* (top-1) e por classe.** Fração de quadros em que a classe predita coincide com a anotada, calculada quadro a quadro (não em janelas isoladas). Reportamos

também a acurácia por classe (*recall*), porque a classe majoritária `no_action` domina e mascara o desempenho nas classes de ação – a mesma armadilha de desbalanceamento discutida nos relatórios anteriores.

- **Contexto ideal vs. ao vivo (*teacher-forced*).** Toda métrica ao vivo é comparada com o mesmo *pipeline streaming*, porém alimentado com o vetor de contexto *correto* (reconstruído das anotações). A diferença isola o custo de construir o contexto ao vivo do desempenho bruto do modelo.
- **Recuperação da sequência do plano.** As confirmações do modelo formam uma sequência de ações (ex. `CCCCSS...`); a “sequência real” é a ordem das intenções acionáveis anotadas. Reportamos a razão *maior subsequência comum / n^o de ações reais* (LCS), que mede quanto do plano o modelo recuperou, e o número de *erros de transição do PlanGraph* (confirmações impossíveis no estado corrente, que o grafo rejeita – ao vivo, isto lançaria exceção).

3 Protocolo

3.1 Dados e variante

Mesma base dos relatórios anteriores: 8 sessões, 2 participantes, roteiros R01/R06 (ordem de montagem invertida) mais R02/R03. O *split* por sessões inteiras é o mesmo: **teste** em S02_20260712 e S05_20260712 (participante/roteiro não vistos no treino), **treino** nas outras seis. A variante avaliada é a **V2 com contexto 10D** (o melhor teto de desempenho dos relatórios anteriores), *seed* 0. O pré-processamento de pose é o *cru* do `skeleton.pkl`, idêntico ao usado no treino dos *checkpoints* (a normalização do `run_webcam_context.py` foi calibrada para o *pipeline* original, de escala diferente, e degrada estes *checkpoints*).

Como a predição é ao vivo, o modo de restrição usado é `no` (argmax dos *logits* com contexto injetado). O corte por entropia (ood), analisado nos relatórios anteriores, suprime justamente as predições de ação que queremos observar aqui, então não é usado na malha; sua recalibração continua sendo a recomendação daqueles relatórios.

3.2 Dois ajustes mínimos, baseados no repositório original

A malha fechada só funciona com dois ajustes sobre a lógica do `run_webcam_context.py`, ambos justificados:

1. **begin_four_tubes como evento externo gravado.** O estágio manual *four_tubes* (entrega de quatro tubos curtos) é disparado ao vivo por um comando de voz do operador, não pela visão. Nas sessões esse evento já está anotado em `plan_events.json`; nós o aplicamos no seu quadro, como substituto elegante do comando de voz. É a *única* informação externa à visão – todas as intenções continuam vindo do modelo.
2. **Reiniciar `old_intention` no repouso.** A guarda `old_intention` do `run_webcam_context.py` impede confirmar a mesma intenção duas vezes seguidas (para não repetir comandos ao robô). Mas a tarefa é feita de sequências repetidas da mesma intenção (`connectors`×4, `screws`×8...), cada uma num alcance discreto separado por `no_action`. Reiniciamos `old_intention` quando o modelo prevê `no_action` (pessoa voltou ao repouso), permitindo uma confirmação por alcance – exatamente o que o *PlanGraph* precisa. *Sem* este ajuste, o contexto trava após a primeira confirmação (1 confirmação contra 24 reais) e o plano desincroniza.

A janela de confirmação `send_window` é um terceiro parâmetro, ajustado empiricamente na Seção 4.1.

3.3 Exclusão dos trechos ignore

Cada sessão tem blocos rotulados `ignore`: o *setup* inicial e final e, sobretudo, o período de entrega manual dos quatro tubos curtos. Esses trechos não são momentos de predição de intenção, e o estado do estágio *four_tubes* durante eles já é atualizado pelo evento externo de voz (item 1 acima). Deixar a visão *confirmar* intenções ali gera confirmações espúrias que corrompem o contexto – foi a maior fonte de erros de transição do **PlanGraph** numa versão preliminar desta análise. Por isso, nos quadros `ignore` a visão é pausada (sem confirmação) e nenhum quadro `ignore` entra em qualquer métrica (acurácia, matriz de confusão, deriva temporal). Esta é a exclusão dos trechos `ignore` adotada em todo o relatório.

4 Resultados

4.1 Ajuste da janela de confirmação

`send_window` é o número de predições iguais consecutivas exigidas antes de confirmar uma intenção e mutar o contexto. O padrão do `run_webcam_context.py` é 3. Aumentá-lo filtra os picos espúrios de predição, ao custo de eventualmente perder alcances muito curtos. A Tabela 1 varre `send_window` no conjunto de teste.

Tabela 1: Varredura de `send_window` no teste (S02, S05), V2(10D), contexto ao vivo. “espúrias” = confirmações disparadas com a pessoa em `no_action`; “erros plano” = transições rejeitadas pelo **PlanGraph**; “seq.” = recuperação da sequência (LCS); “acc. fim” = acurácia no terço final da sessão.

<code>send_window</code>	top-1	connectors	screws	wheels	confirm./real	espúrias	erros plano	seq.
3	63,9%	0,79	0,89	0,51	54/48	20	17	0,79
4	63,9%	0,79	0,89	0,51	53/48	17	17	0,79
5	69,2%	0,79	0,91	0,62	56/48	18	2	0,88
6	68,7%	0,81	0,92	0,62	55/48	17	1	0,88
8	68,5%	0,82	0,92	0,62	51/48	13	0	0,88
10	71,6%	0,72	0,78	0,61	40/48	9	0	0,73

Escolhemos `send_window=8`: é o melhor equilíbrio – confirmações espúrias e erros de **PlanGraph** praticamente zerados, maior recuperação da sequência, e generaliza para o treino (validado separadamente). Valores maiores (≥ 10) começam a *subcontar* (40 confirmações contra 48 reais) e inflam o top-1 via `no_action`, como mostra a queda de `connectors` (0,72) e `screws` (0,78) na última linha. Todos os resultados a seguir usam `send_window=8`.

4.2 Desempenho ao vivo vs. contexto ideal

A Tabela 2 e a Figura 1 comparam o preditor com o contexto construído ao vivo contra o mesmo *pipeline* com o contexto ideal (*teacher-forced*), no conjunto de teste.

O custo agregado de construir o contexto ao vivo é pequeno: **top-1 de 70,9% para 68,5% (−2,4 p.p.)**. Mas a leitura por classe é mais informativa e revela onde o preço é pago:

Tabela 2: Acurácia *streaming* no teste (S02, S05), V2(10D): contexto construído ao vivo pelo preditor vs. contexto ideal. Colunas à direita: acurácia por classe (*recall*). Quadros **ignore** excluídos.

Contexto	Top-1	no_action	connectors	screws	wheels
Ideal (<i>teacher-forced</i>)	70,9%	59,1%	92,3%	98,4%	97,2%
Ao vivo (preditor)	68,5%	61,4%	82,0%	92,4%	61,7%
Diferença	-2,4	+2,3	-10,3	-6,0	-35,5

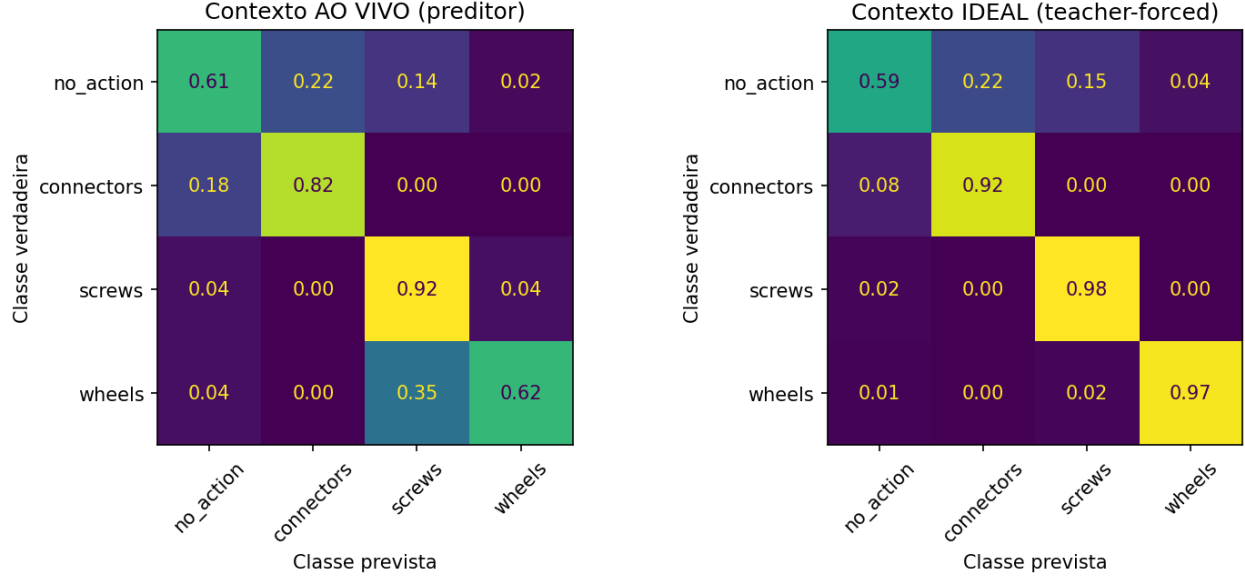


Figura 1: Matrizes de confusão do teste (S02+S05), V2(10D), normalizadas por linha (cada linha soma 1,0), no padrão do **hrc-finetune**. Esquerda: contexto construído ao vivo pelo preditor. Direita: contexto ideal (*teacher-forced*). A diferença mais visível é a linha **wheels**: com contexto ideal o *recall* é 0,97; ao vivo cai para 0,62, com 0,35 da massa vazando para **screws**.

- **get_screws** e **get_connectors** caem pouco (98,4%→92,4% e 92,3%→82,0%): o contexto ao vivo os sustenta bem.
- **get_wheels despenca** (97,2%→61,7%). A Figura 1 mostra que 0,35 dos quadros **wheels** são previstos como **screws**: as rodas vêm no fim da montagem, quando o contexto ao vivo já acumulou a maior deriva (Seção 4.3), e o modelo confunde a fase final de parafusar com a de rodas.
- **no_action** sobe ligeiramente (59,1%→61,4%): parte dos erros de ação vira **no_action**, a classe majoritária – o mesmo efeito de mascaramento discutido nos relatórios anteriores, que é por que a diferença de top-1 (-2,4) subestima a degradação real por classe.

4.3 Escalabilidade

Generalização. O top-1 ao vivo é 68,5% no teste (não visto) contra 73,7% no treino – uma diferença de 5,2 p.p., compatível com a lacuna de generalização já esperada para novos participantes/roteiros com poucos dados (8 sessões, 2 participantes).

Estabilidade temporal. A Tabela 3 divide cada sessão em três terços iguais e mede a acurácia ao vivo em cada um. A queda ao longo da sessão é a assinatura da *deriva* do contexto: como o vetor é cumulativo, os pequenos erros de contagem se acumulam e o contexto do fim da sessão é o mais distante do ideal.

Tabela 3: Acurácia *streaming* ao vivo por terço da sessão (início / meio / fim), V2(10D), `send_window=8`.

<i>Split</i>	Início	Meio	Fim
Teste	72,1%	70,1%	63,3%
Treino	75,2%	74,0%	72,0%

No teste a acurácia cai 8,8 p.p. do primeiro ao último terço. É uma deriva real, porém contida (bem menor que os ~26 p.p. observados numa versão preliminar com `send_window=3`). A conclusão de escalabilidade: **para janelas curtas o contexto ao vivo é confiável; para uso contínuo e sessões longas, a deriva recomenda re-sincronizar o estado por eventos externos** (Seção 5).

4.4 Onde o contexto erra, e o papel do comando de voz

A Tabela 4 mostra o erro médio, dimensão a dimensão, entre o vetor de contexto ao vivo e o ideal (no teste, sobre os quadros com predição). Ela localiza a deriva: as dimensões que dependem da *inferência da visão* (estágio ativo, contagem de parafusos) derrapam mais, enquanto as controladas pelo evento externo de voz (estágio *four_tubes* e seus parafusos) ficam quase perfeitas.

Tabela 4: Erro médio $|\text{ao vivo} - \text{ideal}|$ por dimensão do contexto 10D (teste). Dimensões ordenadas do maior para o menor erro.

Dimensão do contexto	Erro médio
<code>stage:none</code>	0,301
<code>stage:top</code>	0,246
<code>screws_top / 4</code>	0,154
<code>long tubes / 4</code>	0,153
<code>screws_bottom / 4</code>	0,112
<code>short tubes / 8</code>	0,069
<code>wheels / 4</code>	0,066
<code>stage:bottom</code>	0,061
<code>screws_four_tubes / 4</code>	0,007
<code>stage:four_tubes</code>	0,000

Ablação do comando de voz. Para testar diretamente a hipótese de que o comando de voz importa para a contagem de tubos, rodamos a malha *sem* o evento `begin_four_tubes` (visão pura). O resultado confirma a hipótese: sem o evento, o erro do contexto nas dimensões de tubo do estágio *four_tubes* salta – `short tubes` de 0,07 para 0,30 e `screws_four_tubes` de 0,01 para 0,42 –, porque a voz é a única fonte dos quatro tubos curtos daquele estágio. Em top-1 o efeito é menor mas favorável ao evento (68,5% com voz vs. 66,5% sem), coerente com a Tabela 4: como as

dimensões controladas pela voz ficam corretas, o evento ajuda tanto a fidelidade do estado quanto, marginalmente, a predição. **O comando de voz (ou um gatilho externo equivalente) é, portanto, necessário para a parte do contexto que a visão não consegue inferir sozinha.**

4.5 Integridade das confirmações por sessão

A Tabela 5 detalha, por sessão, quão bem a construção ao vivo recupera o plano. Sete das oito sessões recuperam $\geq 0,83$ da sequência real, com zero erros de transição; a exceção é S08, onde um segmento de alcance rotulado **ignore** (não relacionado à entrega de tubos) é pulado e a malha perde o fio – um lembrete de que o rótulo **ignore** é usado para finalidades distintas e nem todo bloco **ignore** é “fora de tarefa”. Note também que em S03 e S04 a acurácia ao vivo *supera* a do contexto ideal: nessas sessões curtas o contexto construído acompanha o plano tão bem quanto o gabarito, e a variância de *streaming* favorece o ao vivo.

Tabela 5: Por sessão (V2(10D), `send_window= 8`): top-1 ao vivo vs. ideal, recuperação da sequência (LCS), erros de PlanGraph e confirmações do modelo por classe. As duas sessões de teste em destaque.

Sessão	Top-1 ao vivo	Top-1 ideal	Seq. (LCS)	Erros plano	Confirm. (C/S/W)
S01	66,5%	72,4%	0,96	0	8/12/5
S02	58,3%	70,6%	0,83	0	8/14/1
S03	80,1%	70,0%	1,00	0	5/4/0
S04	82,1%	75,0%	1,00	0	6/9/0
S05	75,6%	71,2%	0,92	0	13/10/5
S06	73,6%	72,4%	0,96	0	9/11/6
S07	77,3%	86,8%	0,67	1	8/9/0
S08	67,5%	81,2%	0,24	0	4/2/0

A Tabela 6 agrega, no teste, o que o modelo *confirmou* contra o que a pessoa de fato fazia no quadro da confirmação. Ela é a informação mais acionável para o próximo ciclo de treino: fora da diagonal (e a linha `no_action`) estão as confirmações erradas que o treino precisa desambiguar.

Tabela 6: Confusão de confirmações no teste: linha = rótulo real no quadro da confirmação; coluna = intenção confirmada pelo modelo. V2(10D), `send_window= 8`.

real \ confirmado	<code>no_action</code>	<code>connectors</code>	<code>screws</code>	<code>wheels</code>
<code>no_action</code>	0	11	2	0
<code>get_connectors</code>	0	10	0	0
<code>get_screws</code>	0	0	19	1
<code>get_wheels</code>	0	0	3	5

Dois padrões se destacam: (i) **treze confirmações espúrias durante o repouso** (linha `no_action`: 11 como `connectors`, 2 como `screws`) – o modelo dispara intenção com a pessoa parada; e (ii) **confusão `wheels`→`screws`** (linha `get_wheels`: 3 de 8 confirmadas como `screws`), coerente com o colapso do *recall* de rodas na Figura 1.

5 Discussão e recomendações

A construção ao vivo do contexto *funciona* com os ajustes desta análise: o custo em top-1 é de apenas 2,4 p.p. no teste, a sequência do plano é recuperada em $\geq 0,83$ na maioria das sessões e os erros de transição do **PlanGraph** são zero no teste. As fragilidades que restam são concretas e sugerem intervenções específicas, em ordem de impacto medido:

1. **Confirmações espúrias no repouso são o erro nº 1** (Tabela 6). Um portão de movimento/confiança antes de confirmar e, sobretudo, *exemplos negativos difíceis* de **no_action** no treino atacariam a causa. Foi por isso que aumentar **send_window** (que filtra picos isolados) teve o maior impacto isolado (Tabela 1).
2. **Deriva temporal $\Rightarrow$ re-sincronização externa.** A queda de acurácia no fim da sessão (Tabela 3) vem do acúmulo de erro no contexto cumulativo. Re-sincronizar o estado por eventos externos confiáveis (o robô sabe o que entregou; a voz sinaliza mudanças de estágio), em vez de deduzir todo o estado só da visão, limita essa deriva. A ablação da Seção 4.4 mostra que os eventos externos já disponíveis (**begin_four_tubes**) mantêm as dimensões que controlam praticamente sem erro.
3. **A classe **get_wheels** precisa de reforço.** Seu *recall* ao vivo (62%) é o mais frágil, por confusão com **get_screws** na fase final (Figura 1, Tabela 6). Mais dados de rodas e/ou realce do contexto de estágio final no treino ajudariam a separá-las.

Vale registrar que a exclusão dos trechos **ignore** (Seção 3.3) foi importante para a limpeza da análise: numa versão preliminar, as confirmações disparadas durante a entrega manual de tubos eram a maior fonte de erros de transição do **PlanGraph**. Ao mesmo tempo, **S08** mostra que o rótulo **ignore** cobre situações heterogêneas; uma revisão da anotação (separar “*setup*/entrega manual” de “alcance não pontuado”) tornaria tanto o treino quanto esta avaliação mais precisos.

6 Conclusão

O modelo treinado *offline* sustenta-se em malha fechada: com o comando de voz reproduzido pelo evento gravado e a janela de confirmação ajustada de 3 para 8, o contexto construído ao vivo pelo próprio preditor custa apenas 2,4 p.p. de top-1 no teste em relação ao contexto ideal, com recuperação da sequência do plano de $\geq 0,83$ na maioria das sessões e zero erros de transição do **PlanGraph** no teste. A escalabilidade tem duas ressalvas mensuráveis: uma deriva temporal (a acurácia cai ao longo da sessão, à medida que o contexto cumulativo acumula erro) e a fragilidade da classe **get_wheels** (confusão com **get_screws** na fase final). As três recomendações – endurecer a confirmação contra o repouso (com *hard negatives* de **no_action**), re-sincronizar o estado por eventos externos confiáveis, e reforçar a classe de rodas – endereçam diretamente os erros medidos, e todas são compatíveis com a infraestrutura já existente do projeto.

Referências

- [1] P. Yu, A. Abuduweili, R. Liu, C. Liu. *Robustifying Long-term Human-Robot Collaboration through a Multimodal and Hierarchical Framework*. arXiv:2411.15711, 2025.